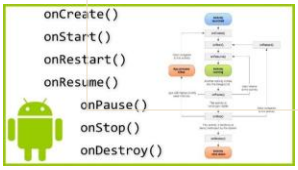
1

Chapitre 4

Activités Android (à base d'XML Views)

Dorra DHAOU



Extrait du cours de : Dr. Naoufel KHAYATI

AU. 2026 - 2027

1

Introduction

2

- Une activité peut être assimilée à un écran qu'une application propose à son utilisateur (**Une activité = un écran d'une application**).
- Pour chaque écran de l'application, on doit donc créer une activité.
- La transition entre deux écrans correspond au lancement d'une activité (via des intents) ou au retour sur une activité placée en arrière-plan.

2

Introduction

3

Composition d'une activité (XML Views)

- Une activité à base d'XML Views est composée de deux volets:
  - **Sa logique métier** (le code Java/Kotlin) et la gestion de son cycle de vie qui sont implémentées dans une classe héritant de Activity (ou de l'une de ses sous-classes).
  - Son **interface utilisateur** définie dans un fichier XML placé dans les ressources de l'application (dans res/layout).
  - Exemple : MainActivity.kt contient le code, et activity_main.xml contient la mise en page.

3

PLAN

4

- **Cycle de vie des activités**
  - Pile d'Activités
  - Cycle de vie
  - Sauvegarde de l'état d'une activité
- **UI des activités**
  - Views
    - Manipulation des objets d'interfaces
    - Gestion des évènements
  - Layouts
    - Types de layouts
    - Configuration du ConstraintLayout

4

5

Cycle de vie des activités

(Valable aussi pour les activités Jetpack Compose)

5

Pile d'activités

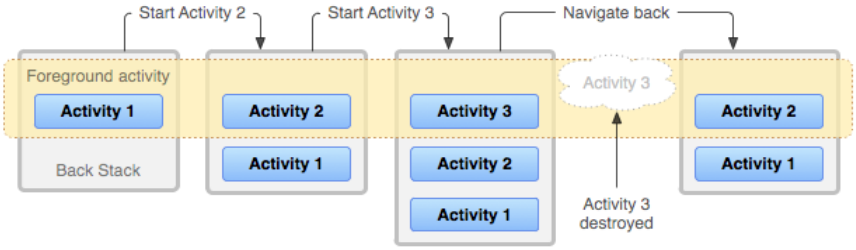
6

- Si un utilisateur reçoit un appel, il devient plus important qu'il puisse y répondre que d'écouter la musique ou de naviguer sur le web.
- Les développeurs d'Android ont pris deux décisions :
  - À tout moment l'application peut laisser sa place à d'autres priorités.
    - Si l'application (en background) utilise trop de ressources empêchant le système de fonctionner correctement, Android pourra décider de l'arrêter sans prévenir.
  - L'activité connaitra plusieurs états au cours de son cycle de vie (active, en pause, etc.).
- Les activités d'une application sont planifiées en utilisant la notion de **pile**.
  - Exp : Quand une activité se lance, elle se met tout en haut d'une pile dite « **pile d'activités** » (*Activity Stack*).
  - Une **pile** est une structure organisée où les éléments sont rangés de manière à respecter le principe : **LIFO** = *Last In, First Out* (*le dernier élément entré est le premier à sortir*).

6

Pile d'activités

7



- Chaque nouvelle activité d'une application s'ajoute comme un élément à la pile.
- Lorsque l'utilisateur appuie sur le bouton « Retour », l'activité en cours est détruite et l'activité précédente reprend (celle qui était dessous réapparaît).
- Android peut tuer une activité en arrière-plan si elle consomme trop de mémoire.

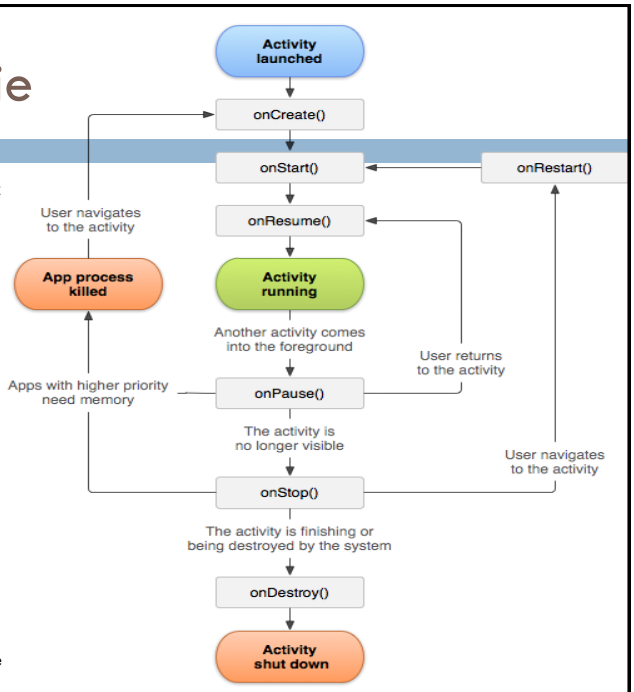
http://developer.android.com/guide/components/tasks-and-back-stack.html

7

Cycle de vie

8

- **onCreate()** : au lancement de l'app. Sert à *initialiser* l'écran
- **onStart()** : activité sur le point d'être visible.
- **onResume()** : activité au premier plan, prête à interagir avec l'utilisateur.
- **onPause()** : une autre activité arrive devant (appel, notification). L'activité actuelle passe en arrière-plan.
- **onStop()** : activité complètement invisible. On doit libérer les ressources.
- **onRestart()** : quand une activité arrêtée redémarre.
- **onDestroy()** : activité détruite (fermeture complète ou décision du système).



http://developer.android.com/reference/android/app/Activity.html

8

Cycle de vie

```kotlin
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
    override fun onStart() {
        super.onStart()
    }
    override fun onResume() {
        super.onResume()
    }
    override fun onPause() {
        super.onPause()
    }
    override fun onStop() {
        super.onStop()
    }
    override fun onDestroy() {
        super.onDestroy()
    }
    override fun onRestart() {
        super.onRestart()
    }
}
```

- Ce code est une **classe MainActivity** écrite en Kotlin pour une application Android.
- C'est l'écran principal (Activity) de l'application qui gère le cycle de vie de l'interface utilisateur.
- Pour l'instant, cette classe ne fait qu'appeler les méthodes parentes (via **super.**) sans ajouter de fonctionnalités personnalisées. C'est une structure de base prête à être enrichie.

Les méthodes principales :

- **onCreate()** : Initialise l'écran au lancement
- **onStart()** : L'écran devient visible
- **onResume()** : L'écran est actif et interactif
- **onPause()** : L'écran perd le focus
- **onStop()** : L'écran n'est plus visible
- **onDestroy()** : L'écran est fermé définitivement
- **onRestart()** : Redémarrage après un arrêt

9

Cycle de vie

10

onCreate()

- Exécutée quand l'utilisateur clique sur l'icône de l'application.
- Sert à créer l'activité en initialisant l'interface graphique.
- Reçoit toujours en paramètre un objet **Bundle** qui contient l'état dans lequel était l'activité avant l'invocation.

onStart()

- Appelée avant que l'activité ne soit visible (début du passage au premier plan / activité sur le point d'être visible).
  - Si l'activité apparaît en premier plan, c'est **onResume()** qui est invoquée.
  - Si l'activité ne peut pas aller en avant plan quelle que soit la raison, l'activité sera transférée à **onStop()**.

10

Cycle de vie

11

onResume()

- Appelée au moment où l'application passe en avant plan.
- À la fin de l'appel à la méthode **onResume()**, l'application se trouve au premier plan et reçoit les interactions utilisateurs.

onPause()

- Appelée lorsque l'activité passe en arrière plan (une autre activité prend sa place en avant plan).
- Une fois cette méthode exécutée, Android peut tuer à tout moment l'activité sans prévenir.
- La nouvelle activité ne démarrera pas tant que l'exécution de cette fonction n'est pas terminée.

11

Cycle de vie

12

onStop()

- Appelée quand l'activité n'est plus du tout visible quelle que soit la raison.
- Dans cette méthode, on doit arrêter tous les traitements et services exécutés par l'application.

onRestart()

- Exécutée lorsque l'activité arrêtée redémarre (repasse en premier plan).

onDestroy()

- Appelée quand l'application sera totalement fermée (Processus terminé).
  - Effectue un nettoyage final avant qu'une activité ne soit détruite (arrêt de threads, par exemple).
  - Cela peut se produire soit parce que l'activité se termine (appel de **finish()** dessus), ou parce que le système a décidé de détruire cette instance de l'activité pour économiser de l'espace.
- Toutes les données non sauvegardées seront perdues.

12

Sauvegarde de l'état d'une activité

13

- **Cas de l'appui sur les touches « Back » / « Home »**
  - *« **Back** » pour revenir vers une activité ou application précédente : Android détruit l'activité qui était en cours.*
    - Si on revient vers l'activité quittée, cette dernière est recréée de nouveau, donc rien n'est restauré (Android la recrée à zéro).
  - *« **Home** » pour revenir sur le bureau : Android peut décharger le contenu des éléments d'interface de l'activité quittée afin de* gagner des ressources en mémoire.
    - Si on revient vers l'activité quittée, cette dernière peut avoir perdu les valeurs saisies dans les zones de texte, par exemple.

13

Sauvegarde de l'état d'une activité

14

- **Cas de rotation de l'écran / changement de langue**
  - Android adapte l'interface dans ces deux cas en arrêtant l'activité puis en la redémarrant.
  - Ce redémarrage peut provoquer la perte des changements apportés à l'activité en cours puisqu'il s'agit d'une recréation de l'interface graphique.
  - Android détruit puis recrée l'activité → risque de perte de données si on n'a rien sauvegardé.

14

Sauvegarde de l'état d'une activité

15

- Par défaut, le système conserve l'état de l'UI d'une activité quittée.
  - Lorsque l'utilisateur navigue vers une activité précédente, son interface utilisateur s'affiche de la façon dont il l'a laissée.
- Ceci est possible grâce à l'objet **Bundle** passé en paramètre à la méthode onCreate().
  - Cet objet permet de restaurer les valeurs d'interface d'une activité qui a été déchargée de la mémoire.
  - **Attention !!!** Un tel objet ne peut sauvegarder que le contenu des objets graphiques ayant un identifiant (ID) (ex : contenu d'un EditText).

Exemple : si l'utilisateur tape son prénom dans un champ, puis tourne l'écran → Android peut restaurer ce champ. Mais si c'est une valeur calculée dans le code (comme un score), elle peut disparaître.

15

Sauvegarde de l'état d'une activité

16

- Pour sauvegarder des valeurs intermédiaires (calculées), il faut : (1)
  - Utiliser la méthode ***onSaveInstanceState (Bundle b)*** au moment du changement d'état de l'activité (Arrêt).

onSaveInstanceState(Bundle b) : appelée avant que l'activité ne soit arrêtée → on y sauvegarde nos variables.

```
override fun onSaveInstanceState(outState: Bundle) {
    outState.putString("Nom", monNom)
    outState.putFloat("Moyenne", maMoyenne)
    super.onSaveInstanceState(outState)
}
```

16

Sauvegarde de l'état d'une activité

17

- Pour sauvegarder des valeurs intermédiaires (calculées), il faut : (2)
  - Récupérer ces valeurs lors du redémarrage de l'activité à l'aide de la méthode ***onRestoreInstanceState (Bundle b)***.

onRestoreInstanceState(Bundle b) : appelée au redémarrage → on récupère nos valeurs et on les réaffiche.

```
override fun onRestoreInstanceState(savedInstanceState: Bundle) {
    super.onRestoreInstanceState(savedInstanceState)
    Toast.makeText( context: this,  text: "Les valeurs récupérées sont "+
                    savedInstanceState.getString( key: "Nom") +
                    " et " +
                    savedInstanceState.getFloat( key: "Moyenne") ,
                    Toast.LENGTH_LONG)
                    .show()
}
```

17

Récap.

- Une **activité =** un écran.
- Android gère une **pile d'activités.**
- Le **cycle de vie** = suite d'états (onCreate → onStart → onResume → etc.).
- Android peut **tuer**/**redémarrer** une activité à tout moment → il faut toujours prévoir la sauvegarde de l'état (**Bundle**).

18

19

UI des activités

19

IHM sous Android

20

- L'interface d'une activité est définie **en XML** (dans res/layout).
- Chaque fichier XML correspond à un **layout**.
- Dans le code Kotlin, on peut accéder à ce layout via **R.layout.nom_fichier**.

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

  - Associe la vue graphique à l'activité.

20

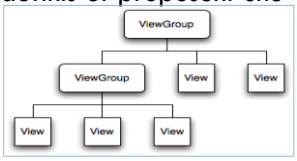
IHM sous Android

21

- Les éléments graphiques sont définis dans le package **android.widget.**
  - Regroupe l'affichage ainsi que la gestion des évènements liés à la zone d'affichage.

```
import android.widget.Button
import android.widget.TextView
import android.widget.RelativeLayout
```

- Une ou plusieurs vues peuvent être regroupées dans un « **ViewGroup** » (Layout / Gabarit).
  - Des groupes de vues particuliers sont prédéfinis et proposent une prédisposition des objets graphiques.



21

IHM sous Android

22

- Deux grandes familles :
- **ViewGroup: (les Layouts):**
  - LinearLayout
  - RelativeLayout
  - FrameLayout
  - TableLayout
  - GridLayout
  - ConstraintLayout
- **View: (les Widgets) :**
  - TextView (Label)
  - EditText
  - Button
  - CheckBox
  - RadioButton
  - ImageButton: Une image qui se comporte comme un bouton.
  - TimePicker / DatePicker: Sélection de l'heure / de date.
  - Etc.

22

Views

23

- TextView

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/entrer_nom"
    android:textSize="@dimen/taille"
    android:textColor="@color/rouge"
    android:layout_alignParentStart="true"
    android:layout_marginStart="41dp"
    android:layout_marginTop="43dp"
    android:id="@+id/textView" />
```

Entrer un Nom:

colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="colorPrimary">#3F51B5</color>
    <color name="colorPrimaryDark">#303F9F</color>
    <color name="colorAccent">#FF4081</color>
    <color name="rouge">#FF0000</color>
</resources>
```

strings.xml

```
<resources>
    <string name="app_name">Chap4_exp</string>
    <string name="entrer_nom">Entrer un Nom:</string>
    <string name="nom">Nom</string>
    <string name="valider">Valider</string>
</resources>
```

dimens.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <dimen name="taille">15sp</dimen>
</resources>
```

23

Views

24

- EditText

```
<EditText
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:hint="@string/nom"
    android:layout_marginTop="32dp"
    android:id="@+id/editText"
    android:layout_below="@+id/textView"
    android:layout_alignStart="@+id/textView" />
```

strings.xml

```
<resources>
    <string name="app_name">Chap4_exp</string>
    <string name="entrer_nom">Entrer un Nom:</string>
    <string name="nom">Nom</string>
    <string name="valider">Valider</string>
</resources>
```

Entrer un Nom:

Nom

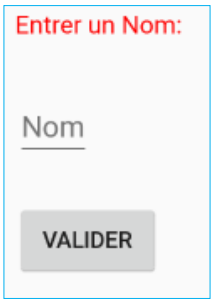
24

Views

25

- Button

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="23dp"
    android:text="@string/valider"
    android:layout_below="@+id/editText"
    android:layout_alignStart="@+id/editText" />
```



strings.xml

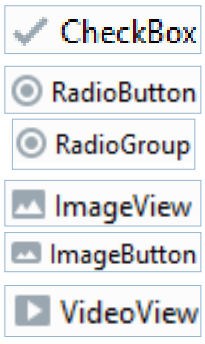
```
<resources>
    <string name="app_name">Chap4_exp</string>
    <string name="entrer_nom">Entrer un Nom:</string>
    <string name="nom">Nom</string>
    <string name="valider">Valider</string>
</resources>
```

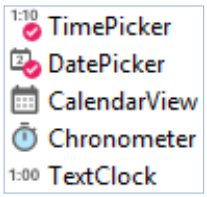
25

Views

26

- CheckBox
- RadioButton
- RadioGroup
- ImageView
- ImageButton
- VideoView
- Widgets pour l'heure et la date





26

Manipulation des Views – **findViewById()**

27

- Les composants graphiques présents dans les layouts sont identifiés par un id. ➔ On identifie chaque composant XML avec un **id**.
  - Ils sont créés dans le fichier xml de layout comme suit : android:id="@+id/textView1"

```
<TextView
    android:id="@+id/textView1"
/>
```

  - Ils étaient principalement accessibles depuis les sources Kotlin et Java via la méthode **findViewById()**
    - Dans KOTLIN : **val tvK = findViewById<TextView>(R.id.textView1)**
    - JAVA : TextView tvJ = (TextView) findViewById (R.id.textView1);
  - Cette méthode est toujours valable, mais elle est couteuse en lignes de code.

27

Manipulation des Views – **View Binding**

28

- Aujourd'hui Google a ajouté la notion de **View Binding** pour remplacer findViewById() aussi bien dans les codes Java que dans les codes Kotlin.
  - Pour activer le **View Binding,** il suffit d'ajouter cette configuration dans le fichier **build.gradle** (niveau module) sous la section *android*.

```
android {

    buildFeatures{
        viewBinding = true
    }

    compileSdk 32
```

28

Manipulation des Views – View Binding

29

- Une fois activé, le View Binding générera automatiquement une classe de liaison pour chacun des fichiers de layout.
  - Nom de la classe de l'activité : **MainActivity**
  - Nom du fichier de layout : **activity_main**
  - Nom de la classe de liaison : **ActivityMainBinding**
    - **Invisible dans l'arborescence du projet.**
- La classe de liaison contient une propriété pour chaque objet d'interface ayant un id.

29

Manipulation des Views – View Binding

30

- Dans l'activité :
  - On récupère l'interface à partir de la classe de liaison.
  - On la stocke dans un objet de liaison
  - On charge le layout de l'objet de liaison avec setContentView(binding.root).
    - L'objet root correspond généralement au ConstraintLayout principal.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val binding = ActivityMainBinding.inflate(layoutInflater)

        setContentView(binding.root)

        binding.
```

```
tvHello                                              TextView
btn1                                                 Button
edPerson                                             EditText
toString()                                           String
to(that: B) for A in kotlin   Pair<ActivityMainBinding, B>
root (from getRoot())                                ConstraintLayout
```

30